Task 0: Research on the BigBird Model

Based on: Zaheer et al., "Big Bird: Transformers for Longer Sequences", NeurIPS 2020 (arXiv:2007.14062).

1. Motivation

BERT-style full self-attention compares every token with every other token, so time and memory grow as **$O(n^2)$**. On common hardware this limits input to about **512 tokens**, which hurts tasks that need long context (QA over documents, long document classification, summarization, genomics).

The paper also asks whether a sparse attention pattern can keep the theoretical power of full attention. The answer is yes: BigBird is a **universal approximator** of sequence functions and is **Turing complete**.

2. Architecture

BigBird is a standard Transformer encoder (the paper builds on RoBERTa and Pegasus checkpoints). Only the **attention pattern** changes. Attention is treated as a graph where token i attends only to its neighbors N(i), not to all tokens. Full attention is the complete graph, and reducing its cost becomes a **graph sparsification** problem.

Each query attends to three kinds of keys:

Component	What it does	Why
Random attention (r keys)	Each query attends to r random keys	Random graphs have short paths between any two nodes, so information spreads fast
Sliding window (w keys)	Each token attends to w/2 neighbors on each side	Language and DNA are local, and neighbors matter most
Global tokens (g tokens)	Attend to everything and are attended to by everything	Act as information hubs. The theory needs them, and ablations show they matter

Full attention (BERT) for one head:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

Here Q K^T is an n × n score matrix, which is the source of the $O(n^2)$ cost.

BigBird (generalized attention): token i only attends to its neighbor set N(i) in the graph D:

$$\text{ATTN}_D(X)_i = x_i + \sum_{h=1}^{H} \sigma\left(Q_h(x_i)\, K_h(X_{N(i)})^T\right) \cdot V_h(X_{N(i)})$$

Q, K, V are the query, key and value functions, σ is softmax, and H is the number of heads. If N(i) contains every token we recover full attention.

The three building blocks written on the adjacency matrix A (A(i,j) = 1 means query i attends to key j):

- Random: A(i, ·) = 1 for r randomly chosen keys.
- Sliding window: A(i, i − w/2 : i + w/2) = 1.

- Global: A(i, :) = 1 and A(:, i) = 1 for every i in the global set G, with g = |G|.

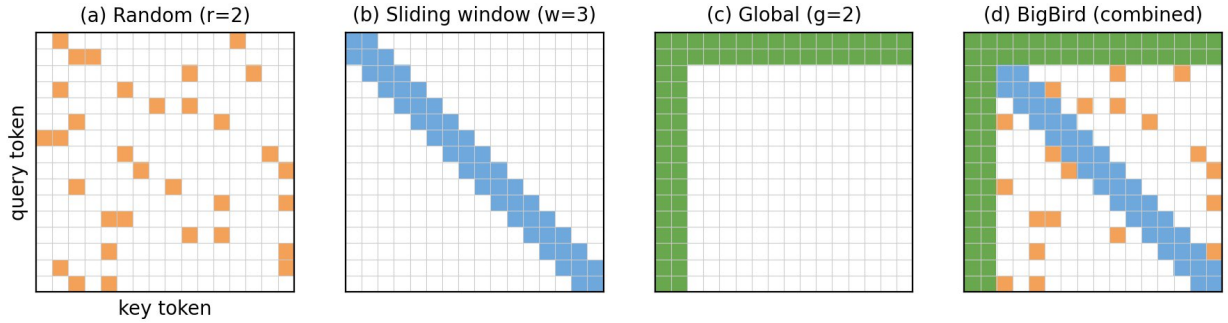


Figure 1. Attention patterns (n = 16 tokens): random, sliding window, global, and the combined BigBird pattern. White = no attention. Drawn for illustration, following Fig. 1 of the paper.

There are two global-token variants:

- **BigBird-ITC** (internal): some existing tokens are made global.
- **BigBird-ETC** (extended): extra tokens (like CLS) are added as global. This performed best on QA.

Together these give the final pattern (Fig. 1d in the paper), which is linear in n. The window plus random pattern alone did not match BERT, so global tokens were needed (Table 1: MLM 62.7 for R+W vs 64.2 for BERT-base).

3. Block-Sparse Implementation

Sparse lookups are slow on GPUs/TPUs, so BigBird works on **blocks** of tokens (block size b, e.g. 64):

- Window attention is computed by "rolling" the key-block tensor.
- Global blocks are always included.
- Random blocks use a gather operation. This is the only inefficient part, so r is kept small.
- Everything is packed into one dense tensor multiplication of cost **O(n(g+w+r)·b·d)**.

$$\text{Cost}_{full} = O(n^2 d) \qquad \text{Cost}_{BigBird} = O(n\,(g + w + r)\,b\,d)$$

Here g, w, r are counted in blocks (g global, w window and r random blocks), b is the block size and d the hidden dimension.

Example (n = 4096, b = 64, ITC setting g = 2b, w = 3b, r = 3b):

- The sequence has 4096 / 64 = 64 blocks.
- Each query block attends to 2 + 3 + 3 = **8 blocks = 512 keys**.
- Full attention uses 4096 keys per query, so BigBird computes only **12.5%** of the scores. The saving grows as n grows.

$$\text{keys per query} = (g + w + r)\,b = (2 + 3 + 3) \cdot 64 = 512 \qquad \frac{512}{4096} = \frac{1}{8} = 12.5\%$$

4. Complexity Comparison

Model	Attention	Complexity
BERT	Full	$O(n^2)$

Model	Attention	Complexity
Sparse Transformer	Fixed sparse	O(n√n)
Reformer	LSH	O(n log n)
Longformer	Window + global	O(n·w), linear
BigBird	Window + global + random	O(n) (≈ O(n(g+w+r)))

BigBird vs Longformer: both are linear. BigBird adds random attention and has a theoretical justification (universal approximation and Turing completeness). Its ETC version uses relative position encodings and trains the global tokens with a CPC loss. Longformer is mostly heuristic.

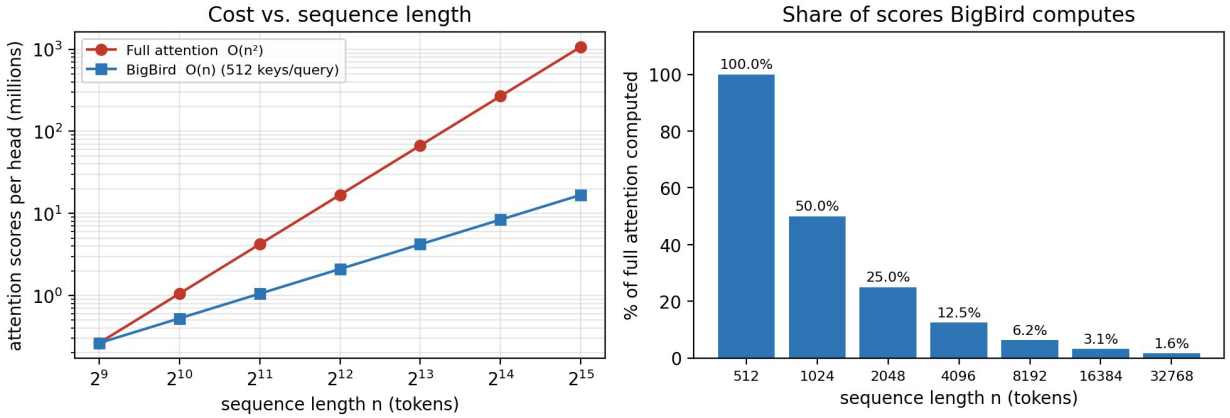


Figure 2. Left: attention scores per head for full attention vs BigBird (512 keys per query). Right: share of the full score matrix BigBird computes. At n = 512 they are equal; the saving grows with n.

5. Suitable Applications for Long Documents

- **Question answering** (HotpotQA, Natural Questions, TriviaQA, WikiHop)
- **Long document classification** (Arxiv, Patents, Hyperpartisan)
- **Long document summarization** (Arxiv, PubMed, BigPatent), with a sparse encoder and a full-attention decoder
- **Genomics** (DNA): promoter-region and chromatin-profile prediction

6. Performance (from the paper)

- **Length:** handles sequences up to **8× longer** (4096 vs 512) on similar hardware.
- **QA (dev, base):** HotpotQA joint 67.8 vs 63.5 (RoBERTa) and 64.4 (Longformer). TriviaQA 79.5 vs 74.3 and 75.2.
- **QA (test, large):** state of the art on Natural Questions (long answer), TriviaQA and WikiHop.
- **Summarization:** BigBird-Pegasus on BigPatent has ROUGE-1 60.64 vs 52.29 for Pegasus.
- **Classification:** Arxiv 92.31 vs 87.96 (previous SoTA), about 5 points better.
- **Genomics:** promoter prediction F1 99.9 vs 95.6 (DeePromoter).

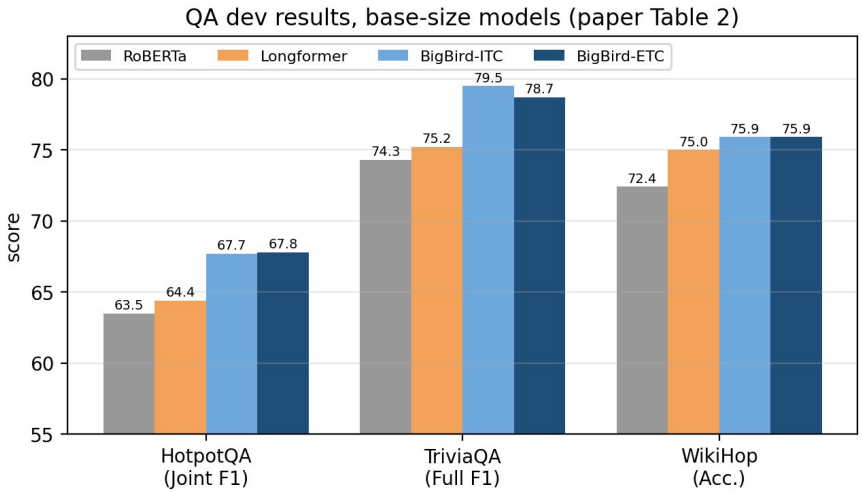


Figure 3. QA dev results for base-size models (values from Table 2 of the paper).

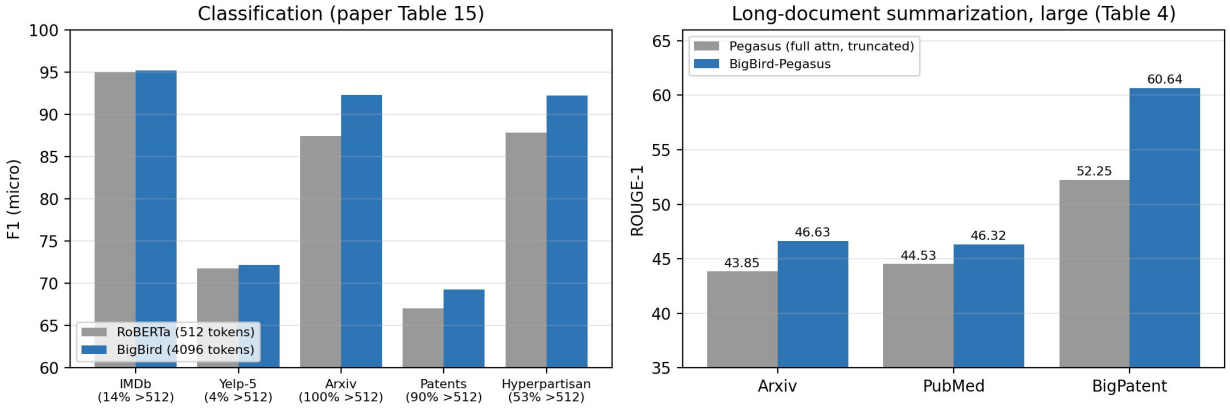


Figure 4. Left: classification F1, with the share of documents longer than 512 tokens in brackets (Table 15). Right: summarization ROUGE-1 for large models (Table 4).

The gains are largest where most documents exceed 512 tokens (Arxiv, Hyperpartisan, BigPatent) and small or absent on short-document datasets (IMDb, Yelp-5).

7. Advantages

- Linear memory and compute in sequence length
- Keeps the theoretical power of full attention
- Hardware-friendly, thanks to blocking
- Large gains on long-context tasks
- Works for both encoder-only and encoder-decoder setups
- Extends beyond NLP to genomics

8. Limitations

- **"No free lunch":** for tasks such as finding the furthest vector for each token, full attention needs O(1) layers, while any sparse attention needs about Ω(n) layers (under the Orthogonal Vectors Conjecture).

Furthest-vector task: for unit vectors u_1..u_n find j = argmax_k ||u_k − u_j||². Full attention solves it in one layer, any sparse attention with Õ(n) edges needs Ω(n^(1−o(1))) layers.*

- **Little or no gain on short inputs:** BigBird is competitive but not better on GLUE (it is below RoBERTa on several tasks), and on short summarization (XSum, CNN/DM) it is about equal to or slightly below Pegasus. IMDb shows no gain.
- Random attention needs gather operations, which are less efficient.
- Extra hyperparameters (block size, g, w, r) need tuning.
- Sparse attention is applied only to the encoder. The decoder stays quadratic, which is fine for short outputs.
- Many results use heavy TPU resources (e.g. 8×8 TPUv3 for pretraining).